Cambrian · 2026-04-01 · retrospective rendering

Gen-16 Failure Diagnosis + Spec Path Fix

Generation 16 passed 68 tests and failed at startup. This record connects a runtime import error to launch instructions, then broadens into a 16-generation analysis.

Real project record · retrospective HTML adaptation

Original text from Cambrian master at a4d2181c488b. The source is preserved in full below and in the linked Markdown. New headings, metadata summaries and visual summaries are an adaptation, not newly observed evidence. Pinned source.

SOURCE AND ADAPTATION

Record provenance

SOURCE RECORD	journal-2026-04-01c	PROJECT	Cambrian
SOURCE DATE	2026-04-01	SOURCE SIGNATURE	2026-04-01T19:15:00+04:00
OPENED TIME	Not recorded	SOURCE AUTHORS	lispmeister + AI assistant
TOOLS (SOURCE)	Claude Sonnet 4.6, Python 3.14.3	STATE (SOURCE)	0d42cbe (cambrian-u3ig), ebb7a1a (journal), as reported
INDEPENDENT WITNESS	Not separately recorded	ADAPTATION	Codex · 2026-09-06 · retrospective
HANDLING	Public repository source; underlying artifacts not imported		

ORIGINAL SOURCE SECTION

Session Goals

1. Diagnose why no `cambrian-base` container runs appeared in `docker ps -a`
2. Understand and document gen-16's failure
3. Fix the root cause in CAMBRIAN-SPEC-005

ORIGINAL SOURCE SECTION

Phase 1: Container Visibility

Question: Running `docker ps -a --filter "status=exited"` showed no containers after the M2 attempt. Where did they go?

Answer: The Supervisor explicitly calls `await container.delete()` in the `finally` block of `run_test_rig()` (`supervisor/supervisor.py:433`). Containers are deleted — not just stopped — after every run. This is by design; `docker ps -a` will never show historical runs.

The artifacts repo (`cambrian-artifacts/`) is the correct place to look for run history, not Docker.

ORIGINAL SOURCE SECTION

Phase 2: Gen-16 Failure Diagnosis

Supervisor logs:

```
2026-04-01T14:29:20 spawn_accepted      generation=16  container_id=lab-gen-16
2026-04-01T14:29:20 test_rig_started    generation=16
2026-04-01T14:29:25 test_rig_complete   generation=16  viable=False
2026-04-01T14:29:26 generation_rolled_back  tag=gen-16-failed
```

Total pipeline time: ~5 seconds. Failure at the `start` stage.

Viability report (`generations.json`, generation 16):

STAGE	PASSED	NOTES
manifest	✓	
build	✓	2165ms
test	✓	68/68 tests passed, 1445ms
start	×	exit code 1, 1008ms
health	—	not attempted

Error (stderr):

```
File "/workspace/src/prime.py", line 53, in run_generation_loop
    from src.loop import generation_loop
ModuleNotFoundError: No module named 'src'
```

Root cause: The start command was `python src/prime.py`. Python adds the script's directory (`/workspace/src`) to `sys.path`, not the workspace root. The generated code used absolute intra-package imports (`from src.loop import generation_loop`), which require `/workspace` to be on `sys.path`. Tests passed because pytest adds CWD to `sys.path` automatically — masking the runtime failure.

The spec was actively steering the LLM into this trap: it said SHOULD NOT use `python -m` and showed `python src/prime.py` in the manifest example.

ORIGINAL SOURCE SECTION

Phase 3: Spec Fix — CAMBRIAN-SPEC-005 v0.14.1 → v0.14.2

Three changes to `spec/CAMBRIAN-SPEC-005.md`:

Change 1: Manifest example `entry.start`

```
- "start": "python src/prime.py",
+ "start": "python -m src.prime",
```

Change 2: Manifest example `files` array

```
- "files": ["src/prime.py", "tests/test_prime.py", "manifest.json", "spec/CAMBRIAN-SPEC-005.md"],
+ "files": ["src/__init__.py", "src/prime.py", "tests/test_prime.py", "manifest.json", "spec/CAMBRIAN-SPEC-005.md"],
```

Without this, LLMs following the example would produce `__init__.py` but omit it from the manifest, breaking `artifact-hash` verification.

Change 3: MUST fields block — extract `entry.start` guidance

Removed the incorrect inline SHOULD NOT warning against `-m`. Replaced with a standalone callout block explaining the `sys.path` semantics:

> `entry.start` **— module form required.** When source lives under a package directory (e.g. `src/`), the start command MUST use `python -m src.prime`, not `python src/prime.py`. The script form adds the script's own directory to `sys.path`, so `from src.loop import …` fails at runtime even though pytest (which adds the working directory to `sys.path` automatically) passes it. The module form adds the working directory (`/workspace`) instead, making both absolute and relative intra-package imports work correctly. The `src/` package directory MUST contain an `__init__.py` file (may be empty).

Change 4: File layout

Added `src/__init__.py` to the SHOULD file layout with a note explaining its role.

Verification: 284 tests passed after the changes (`uv run pytest -v`).

ORIGINAL SOURCE SECTION

Key Observations

- The failure mode (tests pass, runtime fails) is a classic pytest-vs-direct-execution `sys.path` discrepancy. The spec's previous guidance actively caused it.
- The `python -m pkg.module` form is the correct invocation for any multi-module package; the old SHOULD NOT was wrong.
- Gen-16 fitness was otherwise good: 68/68 tests, 1831 source lines, 12 source files — the organism was capable but mis-invoked.

ORIGINAL SOURCE SECTION

Phase 4: Full Viability Data Analysis — Is Python the Problem?

All 16 generations at a glance

GEN	OUTCOME	FAILURE STAGE	ROOT CAUSE
1	promoted	—	
2	failed	test	TypeError in 5 supervisor client tests (mock/type mismatch)
3	failed	test	ParseError message format didn't match test expectation
4	promoted	—	

GEN	OUTCOME	FAILURE STAGE	ROOT CAUSE
5	failed	test	`</file:end>` parser bug — embedded close tags in content
6	promoted	—	
7	failed	test	Same `</file:end>` parser bug as gen-5 (repeated)
8	promoted	—	
9	promoted	—	
10	promoted	—	
11	failed	health	Stale Docker image — report written to `/workspace` not `/output`
12	failed	health	Same stale image (retry)
13	failed	health	Same stale image (retry)
14	failed	build	`/venv` owned by root — permission denied on `uv pip install`
15	promoted	—	
16	failed	start	`ModuleNotFoundError: No module named 'src'` (this session's fix)

Summary statistics

```
Total: 16 generations
Viable: 7 (44%)  — gens 1, 4, 6, 8, 9, 10, 15
Failed: 9 (56%)
```

Failure classification

CATEGORY	COUNT	% OF FAILURES	GENS	PYTHON-SPECIFIC?
Infrastructure (stale Docker, /venv perms)	4	44%	11–14	No — our bug
Logic (genuine test failures)	4	44%	2, 3, 5, 7	Partially
Python structural (import path)	1	11%	16	Yes

Removing infrastructure failures (all the same root cause, now fixed), the LLM-attributable record is:

`12 LLM-generated attempts → 7 viable (58%), 5 failed (42%)`

Logic failure details

Gen-2 (5/59 tests failed): `TypeError` in all 5 `TestSupervisorClientHTTP` tests. Likely a type mismatch in mock setup or response parsing. Pure logic bug — would occur in any language.

Gen-3 (1/63 tests failed): `test_parse_error_message_contains_path` — the parser correctly raises `ParseError("Unclosed <file path='missing_end.py'> block")` but the test expected the path in a different format. Edge case in error message string matching.

Gen-5 & Gen-7 (1/58 tests failed, identical bug): `test_parse_multiple_close_tags_in_content` — the `</file:end>` state machine parser incorrectly consumes close-tag-like text appearing inside file content. The spec requires a state machine (not regex), and the LLM's implementation has a systematic weakness here. **This bug repeated across two independent generations**, suggesting a blind spot in how the LLM understands the parser spec, not a random error.

Where Python specifically hurts us

Even though only 1/5 LLM-attributable failures was directly Python-structural, Python widens the gap between "code that looks correct" and "code that runs correctly in a container" in four specific ways:

1. **No compile step catches structural errors.** In Go or Rust, `from src.loop import generation_loop` with a wrong module path is a *build* failure (stage 2), not a *start* failure (stage 4). The test rig would catch it 2 stages earlier.
2. **Implicit environment magic.** pytest adds CWD to `sys.path`. `python -m` adds CWD. `python script.py` adds the script's directory. `PYTHONPATH` overrides everything. The LLM must navigate these correctly — and the rules are subtle enough that experienced humans get them wrong.
3. **Late binding everywhere.** Imports inside functions (like the `from src.loop import generation_loop` on line 53 of the generated prime.py) aren't evaluated until that code path executes. The test suite exercised the HTTP server but never triggered the generation loop — so the bad import sat undetected. In a compiled language, all imports resolve at build time.
4. **No single artifact.** Go produces a static binary. Python needs an interpreter + venv + installed deps + correct working directory + correct `sys.path` + correct `__init__.py` files. Each is a failure mode the LLM must get right.

Counterarguments for staying with Python

- LLMs generate Python better than any other language (training corpus size). Switching to Go would trade runtime brittleness for more frequent generation errors.
- The spec is 13k tokens of Python-specific guidance. Rewriting for another language is a major M2 undertaking.
- The test rig pipeline is language-agnostic; structural fixes can be added without changing languages.

Recommendations for next run (revisit 2026-04-02)

High-value, low-cost fixes (do before next run):

1. **Smoke-import stage.** Add `python -c "import src.prime"` as a pre-start check in the test rig or as a spec-prescribed test. Catches every import-path failure in <1s. Would have caught gen-16.
2. **Spec clarity on `</file:end>` parser.** Gens 5 and 7 failed with the same parser bug. The spec should include a test vector showing embedded close-tag-like text inside file content. This addresses 2/5 LLM-attributable failures (40%).

Medium-value (consider for M2 hardening):

3. **Pre-compile gate.** Run `python -m py_compile src/*.py` before the test stage. Catches syntax errors and some import issues at build time.

4. **Test coverage for startup path.** The spec's test requirements (§ Tests) don't require testing the generation loop startup path — only the HTTP API. Adding a test that imports and calls the loop entry point would prevent runtime-only failures.

Open question: Will M2 selection pressure solve this organically (organisms with correct imports survive, others don't) or does the failure rate waste too much budget on obviously-fixable structural errors? If the structural failure rate stays >20%, the smoke-import gate pays for itself immediately.

ADAPTATION — NOT HISTORICAL CAPTURE

What the source makes visible

KIND	CREATOR	SUPPORT	NEW MEASUREMENTS
Retrospective source summary	Codex · 2026-09-06	Retained original Markdown	None

Reported sequence and boundary

Manifest: pass → Build: pass · 2165 ms → Tests: 68/68 · 1445 ms → Start: fail · 1008 ms → Health: not attempted

The displayed stage results and logs are retained source excerpts, not rerun results. General claims and recommendations are preserved as author analysis, not independently validated facts. The source says “Three changes” and then lists four; that inconsistency is deliberately retained.

ADAPTATION PROVENANCE

Source evidence and retention

SOURCE	RETENTION	INTEGRITY
Complete original Markdown	Unchanged local bytes	8078ccfc5a781cd13d9181183d742a000e5ffb6afbddb1af05f283d2502e6fea
Pinned upstream record	External; requires network when followed	a4d2181c488bc5f0ff329438541cba51e8543fc8

REVIEW LIMITATIONS

What this rendering cannot establish

The displayed stage results and logs are retained source excerpts, not rerun results. General claims and recommendations are preserved as author analysis, not independently validated facts. The source says “Three changes” and then lists four; that inconsistency is deliberately retained.

Source recommendations remain source recommendations. No new historical observations, predictions, witness signatures or experiment results were created. The original signature is reproduced below as attribution.

ORIGINAL SOURCE CLOSURE

Signature and next entry, as recorded

Signed: lispmeister + AI assistant **Date:** 2026-04-01T19:15:00+04:00

Participants & Tools: Claude Sonnet 4.6, Python 3.14.3 **Commit / Witness:** 0d42cbe (cambrian-u3ig), ebb7a1a (journal)
Related Specs / Beads: CAMBRIAN-SPEC-005 v0.14.2, BOOTSTRAP-SPEC-002 v0.9.1, cambrian-u3ig

Next journal entry: journal-2026-04-02.md (use TEMPLATE.md)

HTML adaptation: Codex · 2026-09-06; historical work not rerun
Unchanged Markdown source